Data Engineering & AI Engineering

یادداشتهای آموزشی، مفاهیم فنی و تمرینهای دوره



پیش‌نیازهای مهندسی داده (۲)

ادامه شبکه، Git, Docker و اتصال به پایگاه داده

جلسه چهارم
مدرس :احمد کریمی
تاریخ :یکشنبه ۲۸ مهر ۱۴۰۵

تاریخ به روز رسانی : 20261019

این جزوه با هدف ثبت، سازماندهی و بازنویسی آموزشی مطالب دوره رایگان »مهندسی داده و مهندسی هوش مصنوعی« تهیه شده است.

محتوای آن بر اساس توضیحات مدرس، مثالهای مطرحشده در کلاس و پرسشوپاسخهای آموزشی تدوین میشود. تلاش شده است مطالب گفتاری کلاس به متنی منظم، قابل فهم و مناسب مطالعه مستقل تبدیل شوند، بدون اینکه مفهوم اصلی صحبتهای مدرس تغییر کند.

هدف این جزوه صرفاً خلاصهکردن جلسات نیست؛ بلکه ایجاد یک منبع آموزشی است که دانشجو بتواند در طول دوره و حتی چند ماه پس از پایان آن، برای یادگیری دوباره مفاهیم، مرور نکات فنی و آمادهشدن برای فعالیتهای عملی به آن مراجعه کند.

در تدوین مطالب، ابتدا مفاهیم اصلی هر جلسه با زبانی ساده و دقیق توضیح داده میشوند. سپس مثالهای مدرس، کاربردهای عملی و ارتباط موضوعات با مسائل واقعی مهندسی داده و هوش مصنوعی بررسی میشوند.

برای حفظ دقت علمی و تمایز میان محتوای اصلی کلاس و توضیحات تکمیلی، از ساختار زیر استفاده شده است:

- مطالب اصلی جلسه :بازنویسی منظم توضیحات مدرس با حفظ مفهوم و مثالهای اصلی.

تکمیل فنی :توضیحات دقیقتر، اصلاح برداشتهای احتمالی و نکات تکمیلی بر اساس منابع فنی معتبر.

- مثال اضافی :مثالها و سناریوهایی که برای درک بهتر مفاهیم به جزوه افزوده شدهاند.
- نکات کاربردی :ملاحظات اجرایی، محدودیتها و تجربههای مرتبط با استفاده عملی از مفاهیم.
- جمعبندی و سوالهای مرور :مرور مهمترین مطالب و ارزیابی میزان یادگیری در پایان هر جلسه.

اصطلاحات تخصصی مانندData Pipeline ، Data Warehouse، ETL، Orchestrationو سایر مفاهیم فنی با نام اصلی انگلیسی حفظ شدهاند تا مطالعه مستندات رسمی و منابع تخصصی برای دانشجویان آسانتر باشد.

این مجموعه بهصورت تدریجی و همزمان با انتشار جلسات جدید تکمیل میشود و دو مسیر اصلی دوره، یعنی Data Engineering وAI Engineering ، را پوشش میدهد.

نسخههای بهروزشده جزوهها، منابع تکمیلی و یادداشتهای هر جلسه از طریق مخزن عمومی GitHub در دسترس خواهند بود. مخزن جزوهها:

https://github.com/LaneZero/data-ai-engineering-course

وبسایت اصلی دوره:

https://mostafaghadimi.github.io/data-course/

یادداشت درباره اعتبار محتوا

این جزوه یک مجموعه آموزشی مستقل و غیررسمی است و وابستگی رسمی به مدرس یا برگزارکنندگان دوره ندارد.

حقوق محتوای اصلی دوره متعلق به صاحبان آن است. توضیحات تکمیلی و مثالهای افزودهشده نیز لزوماً بیانگر دیدگاه یا تأیید مدرس نیستند.

با وجود تلاش برای حفظ دقت فنی، توصیه میشود هنگام اجرای پروژههای واقعی، اطلاعات حساس یا تصمیمگیریهای فنی مهم، مستندات رسمی ابزارها و منابع اصلی نیز بررسی شوند.

هدف از انتشار این مجموعه، کمک به یادگیری عمیقتر، مرور منظم مفاهیم و اشتراک دانش با علاقهمندان حوزه داده و هوش مصنوعی است.

فهرست مطالب

۱.معرفی جلسه و اهداف یادگیری ٤

۲.لایه‌های شبکه و ارتباط میان پروتکل‌ها ٤

۲.۱ TCP و UDP ٥

۳.آدرس‌دهی: IPv4, IPv6 و شبکه خصوصی ٥

۳.۱ DNS: یافتن آدرس مقصد ٦

۳.۲ DHCP: گرفتن تنظیمات شبکه ٦

۴.HTTP: درخواست، پاسخ و API ٦

۴.۱ روش‌های متداول HTTP ۷

۴.۲ کدهای وضعیت HTTP ۷

۴.۳ آشنایی با REST و مستندسازی API ۸

۵.HTTPS و TLS: امنیت ارتباط ۸

۵.۱ بررسی ارتباط HTTP با curl ۸

۶.SSH: اتصال به سرور از راه دور ۹

۷.Docker: از ماشین مجازی تا کانتینر ۹

۷.۱ Image, Repository, Container ۱۰

۷.۲ اجرای PostgreSQL در محیط آزمایشی ۱۰

۷.۳ IP, Port و تفاوت پروتکل سرویس ۱۱

۷.۴ اتصال به PostgreSQL با psql ۱۲

۷.۵ تمرین مرحله‌ای و کنترل نتیجه ۱۲

۸.جمع‌بندی درس ۱۳

۹.واژه‌نامه جلسه چهارم ۱۳

۱۰.سوال‌های مرور ۱٤

۱۱.پاسخ‌نامه مختصر ۱٥

۱.معرفی جلسه و اهداف یادگیری

در جلسه سوم با Linux و مقدمات شبکه آشنا شدیم. جلسه چهارم از همان نقطه ادامه پیدا می‌کند: نخست لایه‌های شبکه و پروتکل‌های پرکاربرد مرور می‌شوند، سپس اتصال امن به سرور و کار عملی با Docker مطرح می‌شود. هدف نهایی مدرس این است که دانشجو بتواند رابطه میان شبکه، سرویس، Container و Database را در یک سناریوی واقعی درک کند.

بعد از مطالعه باید بتوانید تفاوت TCP و UDP، آدرس‌های IPv4 و IPv6, DNS و DHCP را شرح دهید؛ چرخه یک درخواست HTTP و نقش TLS را توضیح دهید؛ کاربرد SSH را بشناسید؛ Image و Container را از هم تشخیص دهید و دلیل استفاده از ابزار مناسب برای اتصال به PostgreSQL را بیان کنید.

مرز جلسه: عنوان درس به ادامه Git اشاره دارد، اما در متن این جلسه آموزش دستوری و عملی Git ارائه نشده است؛ GitHub صرفاً در مثال‌های وب و مخزن کد مطرح می‌شود و مدرس یک منبع آموزشی برای Git معرفی می‌کند. تمرکز عملی اصلی روی Docker و پایگاه داده است. مثال‌های زنده کلاس به‌دلیل خطاهای تشخیص گفتار و نبود تصاویر اسلاید، در مواردی نیازمند تطبیق با ویدیو هستند.

۲.لایه‌های شبکه و ارتباط میان پروتکل‌ها

مدرس یادآوری کرد که مدل OSI شبکه را به هفت لایه تقسیم می‌کند. در مسیر کاری مهندس داده معمولاً شناخت لایه‌های شبکه، انتقال و کاربرد اهمیت بیشتری پیدا می‌کند؛ درگیر شدن با جزئیات فیزیکی کابل و سیگنال لزوماً کار روزانه نیست. پروتکل‌های هر لایه به کمک یکدیگر داده را جابه‌جا می‌کنند و اطلاعات مدیریتی خود را به بسته اضافه می‌کنند.

> **تکمیل فنی:** مدل OSI یک مدل مفهومی هفت‌لایه است، اما اینترنت در عمل معمولاً با مدل TCP/IP توضیح داده می‌شود. فرایند اضافه‌شدن Header در مسیر ارسال را Encapsulation می‌نامند. نام دقیق بسته‌ها و ترتیب Headerها بسته به پروتکل فرق دارد؛ بهتر است آن را صرفاً «چند پاکت تودرتو» تصور نکنیم.

TCP۲.۱ و UDP

در مثال مدرس، پخش زنده و مکالمه آنلاین به جریان داده‌ای نیاز دارند که تأخیر کم در آن مهم است؛ بنابراین همیشه منطقی نیست برای رسیدن هر قطعه منتظر تأیید بمانیم. در مقابل، بارگذاری کامل یک صفحه وب یا ارسال داده‌ای که باید درست و به ترتیب برسد، به قابلیت‌های انتقال قابل‌اعتماد نیاز دارد. از این مثال‌ها برای توضیح شهودی تفاوت TCP و UDP استفاده شد.

ویژگی	TCP	UDP
ارتباط	اتصال‌گرا	بدون اتصال در سطح پروتکل
ترتیب داده	تحویل مرتب جریان بایت در حالت موفق	ترتیب تحویل تضمین‌شده نیست
بازفرستادن داده گم‌شده	دارد	خود پروتکل ندارد
کاربردهای نمونه	بسیاری از ارتباطات وب و پایگاه داده	بخشی از ارتباطات بلادرنگ و DNS

تکمیل فنی: UDP به‌معنای «همیشه مناسب‌تر برای ویدیو» یا TCP به‌معنای «تأیید جداگانه تک‌تک اجزای صفحه» نیست. لایه کاربرد می‌تواند روی UDP سازوکار اطمینان و کنترل ازدحام بسازد؛ HTTP/3 نیز از QUIC روی UDP استفاده می‌کند. انتخاب پروتکل تابع نیاز محصول و پیاده‌سازی است.

۳.آدرس‌دهی: IPv4, IPv6 و شبکه خصوصی

برای یافتن مقصد در شبکه از آدرس IP استفاده می‌شود. مدرس دامنه قابل‌خواندن برای انسان مثل `google.com` را با آدرس‌های عددی شبکه مقایسه کرد. آدرس IPv4 از چهار بخش عددی تشکیل می‌شود و هر بخش از صفر تا ۲۵۵ است. محدودیت فضای آدرس IPv4 یکی از دلایل توسعه IPv6 بوده است.

تکمیل فنی: فضای IPv4 از نظر نظری ۲ به توان ۳۲ آدرس، تقریباً ۴٫۳ میلیارد مقدار دارد، نه حدود یک میلیون. همه این مقادیر برای آدرس‌دهی عمومی آزاد نیستند. IPv6 یک آدرس ۱۲۸ بیتی دارد؛ «IPv8» نسخه جایگزین استاندارد IPv4 نیست و واژه‌ای که در Transcript به آن شبیه است به احتمال زیاد خطای گفتار به متن برای IPv6 است.

آدرس‌های Public برای ارتباط قابل مسیریابی در اینترنت استفاده می‌شوند؛ آدرس‌های Private در شبکه‌های محلی کاربرد دارند. مدرس به شبکه خانگی و آدرس‌های مدیریتی روتر اشاره کرد. مثال‌های متداول محدوده خصوصی شامل `10.0.0.0/8`، `172.16.0.0/12` و `192.168.0.0/16` هستند.

تکمیل فنی: هر آدرسی که با ۱۷۲ شروع می‌شود Private نیست؛ فقط بازه `172.16.0.0` تا `172.31.255.255` در این گروه قرار می‌گیرد. همچنین آدرس Private به‌تنهایی از اینترنت عمومی قابل Route نیست و معمولاً ارتباط خروجی با سازوکاری مثل NAT انجام می‌شود.

DNS۳.۱: یافتن آدرس مقصد

در توضیح کلاس، **Domain Name System (DNS)** سازوکاری است که به برنامه کمک می‌کند برای یک نام دامنه، رکوردهایی مانند آدرس IP پیدا کند. یک دامنه ممکن است بیش از یک آدرس IP داشته باشد؛ پاسخ نیز ممکن است با زمان و مکان درخواست تفاوت کند. فرایند یافتن پاسخ را DNS Resolution می‌نامیم.

تکمیل فنی: DNS فقط تبدیل یک نام به یک IP نیست؛ انواع Record مانند A, AAAA, CNAME, MX و TXT دارد. پاسخ می‌تواند در Cache بماند و Resolver در صورت نیاز به سرورهای دیگر مراجعه کند. خروجی `ping` نیز الزاماً تمام IPهای دامنه را نشان نمی‌دهد.

DHCP۳.۲: گرفتن تنظیمات شبکه

مدرس توضیح داد دستگاه هنگام پیوستن به بسیاری از شبکه‌های محلی، تنظیمات لازم را از شبکه دریافت می‌کند. **Dynamic Host Configuration Protocol (DHCP)** معمولاً آدرس IP و برخی تنظیمات مثل Gateway و DNS را به دستگاه می‌دهد. این تخصیص غالباً با مفهوم Lease یا اجاره زمانی همراه است.

تکمیل فنی: لزوماً لازم نیست دستگاه هر چند دقیقه صرفاً «اعلام زنده بودن» کند. زمان Lease و لحظه‌های تمدید بر اساس تنظیمات DHCP تعیین می‌شوند. برخی شبکه‌ها هم از IP استاتیک استفاده می‌کنند.

HTTP.۴: درخواست، پاسخ و API

مدرس با مثال بازکردن وب‌سایت توضیح داد که مرورگر درخواست می‌فرستد و سرور پاسخ را برمی‌گرداند. یک پیام HTTP معمولاً دارای اطلاعات درخواست یا پاسخ، Header و در صورت نیاز Body است. Header می‌تواند درباره نوع محتوا، قالب پذیرفته‌شده و دیگر ویژگی‌های تبادل داده باشد. Body برای محتوای ارسالی یا دریافتی استفاده می‌شود.

۴.۱روش‌های متداول HTTP

Method	کاربرد رایج	مثال کاربردی
GET	دریافت اطلاعات	مشاهده فهرست کالاها
POST	ثبت داده یا آغاز یک عملیات	ایجاد حساب کاربری
PUT	جایگزینی وضعیت یک منبع	به‌روزرسانی کامل مشخصات
PATCH	تغییر بخشی از منبع	اصلاح نشانی
DELETE	درخواست حذف منبع	حذف یک نشانی

تکمیل فنی: این رفتارها قراردادهای معنایی پروتکل‌اند، نه تضمین اینکه هر سایت الزاماً همین پیاده‌سازی را داشته باشد. GET در حالت متعارف برای خواندن به‌کار می‌رود، اما می‌تواند Header و در بعضی پیاده‌سازی‌ها Body داشته باشد؛ صرف داشتن یا نداشتن Body تعریف GET نیست. PUT و PATCH نیز از نظر معنایی یکسان نیستند.

۴.۲کدهای وضعیت HTTP

خانواده	مفهوم کلی	نمونه
2xx	پاسخ موفق	`200 OK`، `201 Created`
3xx	تغییر مسیر یا پاسخ وابسته به Cache	`301 Moved Permanently`
4xx	اشکال در درخواست یا محدودیت دسترسی	`400 Bad Request`، `404 Not Found`
5xx	خطا در سمت سرور	`500 Internal Server Error`

در کلاس، مثال ثبت‌نام نامعتبر و صفحه پیدانشده برای نشان دادن کاربرد Status Codeها مطرح شد. همچنین مدرس اشاره کرد که Path پس از نام دامنه می‌تواند مشخص کند کدام Resource درخواست شده است.

تکمیل فنی: خانواده ۴xx لزوماً به معنی «اشتباه کاربر» به مفهوم عرفی آن نیست؛ ۴۰۱ برای احراز هویت لازم، ۴۰۳ برای منع دسترسی و ۴۲۹ برای محدودیت نرخ نیز وجود دارد. تفسیر خطای واقعی باید با Body پاسخ و مستندات API همراه باشد.

۴.۳آشنایی با REST و مستندسازی API

مدرس تفاوت وظایف توسعه‌دهنده سمت سرور و کلاینت را با نمونه سرویس راننده و مسافر توضیح داد. در چنین معماری‌ای، API قراردادی میان بخش‌هاست: کلاینت مسیر و اطلاعات مناسب را ارسال می‌کند و سرور عملیات را انجام می‌دهد. ابزارهای مستندسازی نیز روش استفاده از API را روشن می‌کنند.

> **تکمیل فنی:** REST فقط مجموعه روش‌های GET و POST نیست؛ یک سبک معماری با محدودیت‌هایی مانند Statelessness و Uniform Interface است. هر API مبتنی بر HTTP الزاماً RESTful نیست. بحث مفصل REST در Transcript تدریس نشده است.

HTTPS.۵ و TLS: امنیت ارتباط

مدرس با تشبیه «پاکت مهروموم‌شده» توضیح داد که اطلاعات بین مرورگر و سرور ممکن است در مسیر انتقال دیده یا دستکاری شوند. در HTTPS، پروتکل TLS به برقراری ارتباط محرمانه و تأیید اصالت سرور کمک می‌کند. گواهی دیجیتال و زنجیره اعتماد نیز بخشی از سازوکار شناسایی طرف مقابل هستند.

> **تکمیل فنی:** HTTPS به معنی HTTP روی TLS است؛ صرفاً «اضافه کردن حرف S» به نشانی نیست. در روند استاندارد TLS، اعتبار گواهی، نام دامنه، زنجیره اعتماد و مذاکره کلیدهای نشست بررسی می‌شوند. محرمانگی، تمامیت داده و احراز هویت سرور سه هدف مهم‌اند، اما HTTPS به‌تنهایی امن بودن کل برنامه را تضمین نمی‌کند.

در کلاس درباره هشدارهای امنیتی مرورگر هم صحبت شد. ظاهرشدن هشدار گواهی می‌تواند ناشی از تاریخ انقضا، نام دامنه ناسازگار، زنجیره اعتماد نامعتبر یا دخالت در مسیر ارتباط باشد.

> **نکته هشدار:** برای ادامه کار آزمایشگاهی هم هشدار گواهی ناشناس را بی‌دلیل دور نزنید. پیش از واردکردن رمز عبور یا اطلاعات واقعی، نشانی و اعتبار گواهی را بررسی کنید.

۵.۱بررسی ارتباط HTTP با curl

مدرس `curl` را ابزاری برای مشاهده درخواست‌ها و پاسخ‌های وب معرفی کرد؛ این ابزار در Shell به‌ویژه روی سرورهای فاقد مرورگر گرافیکی کاربرد دارد.

```
# Inspect only response headers for a public site
curl -I https://example.com

# Display verbose connection and protocol details
curl -v https://example.com
```

تکمیل فنی: `curl -I` درخواست HEAD می‌فرستد و فقط Header پاسخ را نشان می‌دهد؛ با `-v` جزئیات بیشتری از ارتباط نمایش داده می‌شود. اطلاعات حساس و Tokenهای واقعی را هنگام اشتراک‌گذاری خروجی حذف کنید.

SSH.۶: اتصال به سرور از راه دور

در محیط‌های مهندسی داده، سرویس‌ها اغلب روی Linux Server اجرا می‌شوند و پنجره گرافیکی مستقیمی ندارند. مدرس SSH را به‌عنوان راه معمول اتصال به این سرورها معرفی کرد و به ابزارهایی مانند PuTTY برای Windows اشاره داشت. در نمونه کلاس از یک ماشین مجازی آموزشی استفاده شد، اما جزئیات تنظیمات کلید و اتصال به‌طور کامل آموزش داده نشد.

```
# Replace the placeholder host with your own lab server
ssh student@server.example.com
```

تکمیل فنی: SSH معمولاً اتصال رمزگذاری‌شده و احراز هویت کاربر فراهم می‌کند و به‌صورت پیش‌فرض از پورت TCP 22 استفاده می‌کند؛ اما پورت می‌تواند تغییر کند. داشتن نشانی سرور، حساب مجاز و روش احراز هویت معتبر ضروری است. کلید خصوصی را در GitHub منتشر نکنید.

Docker.۷: از ماشین مجازی تا کانتینر

مدرس برای توضیح ایده کانتینرها، از ماشین مجازی شروع کرد: می‌توان روی یک میزبان چند VM با منابع مجزا ساخت، اما هر VM معمولاً سیستم‌عامل Guest مستقل خود را اجرا می‌کند. در مقابل، Containerها برای ایزوله‌کردن فرایندها از امکانات سیستم‌عامل میزبان استفاده می‌کنند و معمولاً سبک‌تر هستند.

تکمیل فنی: کانتینر و VM از نظر مرز امنیتی و جداسازی یکسان نیستند. کانتینرهای لینوکسی از Kernel میزبان استفاده می‌کنند و قابلیت‌هایی مانند Namespace و cgroup به تفکیک دید و محدودکردن منابع کمک می‌کنند. Docker Desktop در Windows و macOS معمولاً از یک VM لینوکسی زیرساختی هم استفاده می‌کند.

مدرس یک سازمان با چند تیم و سرویس را مثال زد که می‌خواهد مصرف منابع و محیط اجرای آن‌ها را کنترل کند. Docker اجرای بسته‌بندی‌شده سرویس‌ها را ساده می‌کند، اما به‌تنهایی جایگزین طراحی امنیت و شبکه نیست.

۷.۱ Image, Repository, Container

Image بسته‌ای نسخه‌دار از محتوای موردنیاز برای اجرای برنامه است. **Repository/Registry** محلی است که Imageها در آن نگهداری و توزیع می‌شوند. **Container** نمونه اجراشده Image است. مدرس توضیح داد می‌توان Image را با `docker pull` دریافت و با `docker run` کانتینر ایجاد کرد؛ Tag به شناسایی نسخه Image کمک می‌کند.

دستور نمونه	معنی	اصطلاح
`docker images`	الگوی بسته‌بندی‌شده برنامه	Image
`docker pull postgres:16`	دریافت Image	Pull
`docker run ...`	اجرای مستقل یک Image	Container
`docker ps`	کانتینرهای در حال اجرا	Running containers
`docker exec ...`	اجرای فرمان در کانتینر	Exec

تکمیل فنی: یک Image الزاماً «سیستم‌عامل کامل» نیست؛ می‌تواند فقط فایل‌های لازم برنامه و وابستگی‌هایش را داشته باشد. Container نیز یک ماشین مجازی کامل با Kernel مستقل نیست. Tag مانند `16` به نسخه اصلی اشاره می‌کند، اما برای تکرارپذیری سخت‌گیرانه می‌توان Digest دقیق Image را ثابت کرد.

۷.۲ اجرای PostgreSQL در محیط آزمایشی

مدرس سرویس پایگاه داده را نمونه انتخاب کرد: سرویس برنامه از اطلاعات استفاده می‌کند، اما داده در یک Database مدیریت می‌شود. در مثال او، متغیرهای محیطی برای تنظیم اولیه نام کاربری و رمز استفاده شدند؛ `-d` برای اجرای کانتینر در پس‌زمینه بود و دستورات Docker وضعیت آن را نشان می‌دادند.

تکمیل فنی: متن Transcript تمام آرگومان‌های دستور واقعی کلاس را به‌صورت قابل اطمینان ثبت نکرده است؛ دستور زیر **مثال اضافی آموزشی** است، نه بازنویسی دقیق فرمان مدرس. این مثال تنها برای سیستم شخصی و بدون داده حساس طراحی شده است.

```
# Pull a PostgreSQL image for the local lab
# Pin the major image version for a more predictable exercise
docker pull postgres:16

# Create a local-only PostgreSQL service with persistent storage
# LAB_PASSWORD must be set in your current shell before running
# Use an isolated development machine, not a shared production server
docker volume create databurst_pgdata

docker run -d \
  --name databurst-pg \
  -e POSTGRES_USER=student \
  -e POSTGRES_PASSWORD="$LAB_PASSWORD" \
  -e POSTGRES_DB=databurst \
  -p 127.0.0.1:5432:5432 \
  -v databurst_pgdata:/var/lib/postgresql/data \
  postgres:16

# Check container status and recent startup messages
docker ps
docker logs --tail 30 databurst-pg
```

نکته هشدار: در این نمونه، `LAB_PASSWORD` یک متغیر محیطی است که خودتان باید پیش از اجرای دستور مقداردهی کنید؛ مقدار خالی قابل قبول نیست. این شیوه برای آزمایش است. در محیط واقعی، Secret Manager یا سازوکار امن‌تر را جایگزین کنید. Volume برای حفظ داده بعد از حذف کانتینر استفاده می‌شود؛ اجرای پایگاه داده بدون ذخیره‌سازی پایدار خطر ازدست‌رفتن اطلاعات دارد.

مدرس توضیح داد که `docker run` بدون `-d` ممکن است خروجی فرایند اجرا را در ترمینال نگه دارد و با `-d` کانتینر در حالت Detached اجرا می‌شود. `docker ps` فهرست کانتینرهای در حال اجرا را نشان می‌دهد.

تکمیل فنی: Detached Mode به معنی «اجرای یک Daemon جدید داخل کانتینر» نیست؛ فقط Docker CLI را از فرایند درحال‌اجرا جدا می‌کند. پایدارماندن کانتینر به زنده‌بودن فرایند اصلی آن وابسته است.

IP, Port۷.۳ و تفاوت پروتکل سرویس

در کلاس توضیح داده شد چند سرویس می‌توانند روی یک ماشین باشند و با شماره Port و Transport Protocol از هم تفکیک شوند. مدرس برای PostgreSQL از پورت متداول `5432` صحبت کرد. همچنین نشانی Loopback یعنی `127.0.0.1` را به‌عنوان «همین رایانه» معرفی کرد.

تکمیل فنی: بازه شماره پورت از صفر تا ۶۵۵۳۵ است. یک Port در TCP و همان شماره در UDP فضاهای مستقل‌اند؛ اما وجود Port باز تضمین نمی‌کند پروتکل دلخواه روی آن صحبت کند. در Docker، `-p 127.0.0.1:5432:5432` پورت میزبان را فقط روی Loopback منتشر می‌کند، نه روی تمام شبکه.

۷.۴اتصال به PostgreSQL با psql

مدرس توضیح داد که ابزار مخصوص پایگاه داده باید با پروتکل PostgreSQL صحبت کند؛ ابزارهایی مانند `curl` که برای HTTP ساخته شده‌اند، برای اجرای Query روی PostgreSQL مناسب نیستند. ابزار فرمانی `psql` و کلاینت‌های درون Python یا زبان‌های دیگر این نقش را دارند.

```
# Run the database client from inside the existing container
docker exec -it databurst-pg psql -U student -d databurst
```

در محیط `psql` می‌توان پرس‌وجوی آزمایشی اجرا کرد:

```
-- Verify the current database and server time
SELECT current_database(), now();

-- Create a disposable practice table
CREATE TABLE IF NOT EXISTS lab_events (
    event_id BIGSERIAL PRIMARY KEY,
    event_name TEXT NOT NULL
);

INSERT INTO lab_events (event_name) VALUES ('session-04');
SELECT * FROM lab_events;
```

تکمیل فنی: فرمان `docker exec -it` روی کانتینری که در حال اجراست فرمان جدید اجرا می‌کند؛ هدف آن «ورود به ماشین مجازی مستقل» نیست. اگر روی میزبان یا کانتینر مجزا اتصال بزنید، باید Host, Port، نام Database و مجوز شبکه را متناسب با همان مسیر تنظیم کنید.

۷.۵تمرین مرحله‌ای و کنترل نتیجه

۱.با `docker --version` و `docker info` مطمئن شوید ابزار و Engine در دسترس‌اند.

۲.متغیر `LAB_PASSWORD` را در محیط Shell محلی خود تنظیم کنید و مقدارش را منتشر نکنید.

۳.دستورهای بخش PostgreSQL را فقط روی ماشین آزمایشگاهی اجرا کنید.

۴.با `docker ps` وضعیت را بررسی کنید و در صورت مشکل خروجی `docker logs` را بخوانید.

۵.با `docker exec` به `psql` وصل شوید و Queryهای نمونه را اجرا کنید.

۶.داده آزمایشی را بررسی کنید؛ سپس در صورت نیاز کانتینر را متوقف کنید. Volume را فقط وقتی حذف کنید که دیگر به داده نیاز ندارید.

```
# Stop the lab service without deleting the persistent volume
docker stop databurst-pg
```

۸.جمع‌بندی درس

این جلسه با توضیح شبکه شروع شد و به اجرای سرویس پایگاه داده در Docker رسید. برای کار با سیستم داده‌محور، دانستن اینکه نام دامنه چگونه Resolve می‌شود، درخواست HTTP چه ساختاری دارد، TLS و SSH کجا کاربرد دارند و یک Container چگونه یک Database را اجرا می‌کند، مقدمه‌ای عملی برای کارهای بعدی است.

در بخش شبکه، نباید TCP/UDP را به دو برچسب «سریع» و «امن» تقلیل داد؛ در بخش Docker نیز Image را با Container و Container را با VM اشتباه گرفت. در بخش دیتابیس، استفاده از پروتکل و کلاینت مناسب و محافظت از داده و رمز عبور جزو پایه‌ای‌ترین الزامات‌اند.

۹.واژه‌نامه جلسه چهارم

اصطلاح	معادل و توضیح کوتاه
OSI	مدل مفهومی هفت‌لایه شبکه
TCP	پروتکل انتقال اتصال‌گرا و قابل‌اعتماد
UDP	پروتکل دیتاگرامی بدون تضمین تحویل در سطح خودش
Encapsulation	بسته‌بندی داده همراه اطلاعات لایه‌های پروتکل
IPv4	آدرس‌دهی ۳۲ بیتی
IPv6	آدرس‌دهی ۱۲۸ بیتی
Public IP	آدرس قابل مسیریابی عمومی
Private IP	آدرس مورد استفاده در محدوده‌های خصوصی
DNS	سامانه نام دامنه
DNS Resolution	فرایند یافتن رکوردهای نام دامنه
DHCP	تخصیص پویای تنظیمات شبکه
Lease	اعتبار زمانی تخصیص آدرس
HTTP	پروتکل درخواست و پاسخ وب
Status Code	کد وضعیت پاسخ
API	رابط برنامه‌نویسی کاربردی
REST	سبک معماری رابط سرویس

TLS	پروتکل امنیت لایه انتقال
Certificate	گواهی دیجیتال
SSH	اتصال امن به سیستم راه دور
Virtual Machine	ماشین مجازی
Container	محیط ایزوله اجرای فرایند
Docker Image	بسته اجرای برنامه و وابستگی‌ها
Registry	محل نگهداری و توزیع Image
Environment Variable	متغیر محیطی برای تنظیم رفتار برنامه
Detached Mode	اجرای کانتینر بدون اتصال مستقیم خروجی به Terminal
Port	شماره نقطه ارتباطی سرویس
Loopback	نشانی بازگشتی میزبان، مانند `127.0.0.1`
PostgreSQL	سامانه مدیریت پایگاه داده رابطه‌ای
psql	کلاینت خط فرمان PostgreSQL
Volume	ذخیره‌سازی مستقل از چرخه عمر کانتینر

۱۰.سوال‌های مرور

۱.چرا در مهندسی داده، شناخت پروتکل‌های شبکه ضروری است؟

۲.مهم‌ترین تفاوت TCP و UDP در تضمین تحویل چیست؟

۳.Encapsulation به چه معناست؟

۴.IPv4 و IPv6 از نظر اندازه آدرس چه تفاوتی دارند؟

۵.تفاوت Public IP و Private IP چیست؟

۶.DNS و DHCP چه وظیفه متفاوتی دارند؟

۷.HTTP Request و Response چه اجزایی دارند؟

۸.GET و POST معمولاً برای چه کاربردهایی استفاده می‌شوند؟

۹.Status Codeهای 2xx و 4xx و 5xx به‌طور کلی چه مفهومی دارند؟

۱۰.HTTPS از TLS برای تحقق کدام هدف‌ها کمک می‌گیرد؟

۱۱.SSH در مدیریت Linux Server چه کاربردی دارد؟

۱۲.تفاوت Image و Container و VM چیست؟

۱۳.تفاوت `docker pull`، `docker run` و `docker ps` چیست؟

۱۴.چرا برای اجرای Query در PostgreSQL، `psql` مناسب‌تر از `curl` است؟

۱۵.در نمونه آزمایشی Docker، محدودکردن Port به Loopback و استفاده از Volume چه فایده‌ای دارد؟

۱۱.پاسخ‌نامه مختصر

۱.برای عیب‌یابی اتصال‌ها و فهم رفتار سرویس‌ها و پایگاه داده.

۲.TCP تحویل مرتب و بازفرستادن در سطح انتقال دارد؛ UDP ذاتاً تضمین نمی‌کند.

۳.افزودن اطلاعات Header و ساختار لایه‌ها هنگام ارسال داده.

۴.IPv4 برابر ۳۲ بیت و IPv6 برابر ۱۲۸ بیت است.

۵.Public در اینترنت Route می‌شود؛ Private برای فضای شبکه خصوصی تعریف شده است.

۶.DNS نام را به رکورد شبکه Resolve می‌کند؛ DHCP تنظیمات را به کلاینت اختصاص می‌دهد.

۷.اطلاعات شروع پیام، Header و Body اختیاری.

۸.GET برای بازیابی؛ POST معمولاً برای ارسال داده یا ایجاد عملیات.

۹.2xx موفقیت، 4xx مشکل مرتبط با درخواست و 5xx خطای سمت سرور.

۱۰.محرمانگی، تمامیت داده و احراز هویت سرور.

۱۱.فراهم‌کردن دسترسی رمزگذاری‌شده و مدیریت از راه دور.

۱۲.Image الگوست؛ Container اجرای آن؛ VM سیستم‌عامل Guest جداگانه دارد.

۱۳.دریافت Image، ساخت/شروع کانتینر و دیدن کانتینرهای فعال.

۱۴.چون PostgreSQL پروتکل اختصاصی دارد و `curl` کلاینت HTTP است.

۱۵.Loopback دسترسی از بیرون میزبان را محدود می‌کند و Volume به ماندگاری داده کمک می‌کند.